
SELF-SUPERVISED REPRESENTATION LEARNING WITH SIMSIAM

PROJECT REPORT

Department of Computer Science
Aarhus University

Department of Computer Science
Aarhus University

December 5, 2025

ABSTRACT

We explore the differences between supervised (SL) and self-supervised (SSL) representation learning for image classification using a SimSiam, a method proposed by [Chen and He \(2020\)](#) utilizing Siamese networks. We implement SimSiam as well as standard cross-entropy SL and observe how they each are affected by various conventional methods for training convolutional neural networks (CNNs). We use CIFAR10 as our dataset. We also investigate related findings of [Chen et al. \(2020\)](#) for SimCLR, another Siamese SSL method bearing resemblances to SimSiam, and test if they apply to SimSiam. As [Chen et al. \(2020\)](#) find for SimCLR, we find that SimSiam requires data augmentations stronger than those beneficial to SL and significantly longer training. Unlike SimCLR, our findings do not suggest that scaling the underlying encoder architecture closes the performance gap between SL and SimSiam. Additionally, we investigate the effects of BN in SimSiam and find that it is an essential component to avoid collapse and facilitate productive training, and we propose a potential explanation taking into account the unique training dynamics specific to SimSiam. We also discover that it is possible to massively simplify the architectural setup of SimSiam in terms of model size without significantly affecting performance, specifically by omitting the projector.

Keywords Visual Recognition · Representation Learning · Self-Supervised Learning · Siamese Networks · SimSiam

1 Introduction

Due to the high computational cost and data demand of training encoders from scratch as well as the often high transferability of learned features, transfer-learning is a widely adopted technique for implementing neural networks in an image recognition setting. At the same time, acquiring labels is a slow, labor-intensive and expensive process, and the amount of unlabeled data available today is much larger than the available labeled data. Self-supervised representation learning is the field of study that addresses the intersection of these two problems: Training encoders to learn generally useful features that transfer well to downstream tasks, while using data that is entirely unlabeled. In recent years, many new methods have been proposed for self-supervised representation learning, all capable of producing results that are competitive with the traditional SL approach, and many with a shared commonality in the use of Siamese networks. In the words of [Chen and He \(2020\)](#) "Siamese networks are weight-sharing neural networks applied on two or more inputs", where typically multiple inputs include augmented views of the same original input. In this way Siamese networks are especially suited for modeling invariance and thereby learning useful representations through a training objective where different views of the same image are embedded closely together in the learned space. The main pitfall with Siamese networks is that of collapsing solutions, where the training objective is trivially satisfied by mapping all inputs to a constant, and the main way in which Siamese methods differ is namely how they prevent collapse. SimSiam is a very simple Siamese method where collapse is prevented by a stop-gradient on one of the branches in the network, and is the one we have chosen to study for this project. We specifically investigate how SimSiam differs from SL in practice, and how common methods for training networks affect the two respectively. Most common methods for training deep CNNs for visual recognition tasks have been developed and proposed from an SL setting, and since the training dynamics of SimSiam vastly differ from those of SL, there is reason to believe that these methods could have different effects when applied to SimSiam.

2 Related Work

SimSiam can be viewed as one member of a family of Siamese SSL methods. Another member of this family is SimCLR (Chen et al. [2020]), which repulses views of different images in the training objective in order to prevent a collapse. This approach is also referred to as contrastive learning, and the different images that must be repulsed are referred to as negatives. SimSiam is not a contrastive method, yet still it shares many similarities with SimCLR in how it functions, specifically in the use of augmented views and the overall Siamese architecture. In the words of Chen and He [2020], SimSiam can be thought of as "SimCLR without negatives". Other Siamese SSL methods include SwAV (Caron et al. [2021]), which uses online clustering, and BYOL (Grill et al. [2020]), which uses a momentum encoder. Chen et al. [2020] contributed multiple findings in how SimCLR compares to SL in terms of its benefit from strong composed data augmentations, learnable nonlinear transformations between the encoder and final loss calculation, larger batch sizes, longer training as well as deeper and wider encoder architectures. Chen and He [2020] show that SimSiam, as opposed to SimCLR, does not require large batch sizes, which begs the question of which of the findings of Chen et al. [2020] apply mainly to contrastive methods, and which apply mainly to Siamese methods. In this project we therefore seek to test these findings and see if they also hold for SimSiam, as well as explore other differences between SL and SimSiam that we encounter along the way.

3 Methods

3.1 Dataset

We use the CIFAR10 dataset (Krizhevsky and Hinton [2009]). The CIFAR-10 dataset consists of 60000 32×32 color images in 10 classes. The 10 different classes represent airplanes, cars, birds, cats, deer, dogs, frogs, horses, ships, and trucks. With 6000 images per class, there are no class imbalances. There are 50000 training images and 10000 test images. CIFAR10 is widely used in the research community and we chose it in particular because of the small size of the images, which enables us to feasibly train encoders from scratch given our time constraints and available hardware. Another benefit of using CIFAR10 is that we were able to reuse hyperparameter settings from Chen and He [2020] and He et al. [2015] for SimSiam and SL respectively and achieve good baseline results without having to perform extensive hyperparameter tuning.

3.2 Supervised Learning

We use a standard SL setup where the d -dimensional embedding produced by the encoder is put through a linear layer that outputs a 10-dimensional vector with an entry corresponding to each class, which is then input to a softmax function to produce a vector of class probabilities. This vector of class probabilities is then used to compute the cross-entropy loss with the true labels as the training objective.

3.3 SimSiam

SimSiam works by first taking two randomly augmented views x_1 and x_2 from each image x in the current batch. The two views are fed to an encoder followed by a projector (together denoted f), typically a multi-layer perceptron (MLP), which produces two embeddings $z_1 = f(x_1)$ and $z_2 = f(x_2)$. Next a predictor, also typically an MLP, denoted h , receives first z_1 and tries to predict z_2 by outputting $p_1 = h(z_1)$ such that the negative cosine similarity of p_1 and z_2 is minimized, and vice versa for $p_2 = h(z_2)$ and z_1 . I.e. let:

$$\mathcal{D}(p_1, z_2) = -\frac{p_1}{\|p_1\|_2} \cdot \frac{z_2}{\|z_2\|_2} \quad (1)$$

The important component to prevent a collapse is to introduce a stop-gradient, which kills the gradient flow along the non-predicting branch in the network by simply setting the gradient of the loss with respect to the projections to zero. This then gives the symmetrized SimSiam loss for an image:

$$\mathcal{L}(x) = \frac{1}{2} \mathcal{D}(p_1, \text{stopgrad}(z_2)) + \frac{1}{2} \mathcal{D}(p_2, \text{stopgrad}(z_1)) \quad (2)$$

Which is averaged across all images in the batch. Its value lies in the range $[-1, 1]$. It is evident that SimSiam (as well as other related Siamese methods) is fundamentally reliant on data augmentation, and it is the responsibility of the practitioner to create a data augmentation pipeline such that the label-determining features of the image are preserved, while irrelevant features are varied.

3.4 Evaluation Metrics

In this project specifically our goal, or downstream task, is to learn a representation space (also referred to as features) that allows for high accuracy multiclass classification on the CIFAR10 dataset. Generally when evaluating the quality of learned features in SSL research, the top-1 accuracy of a linear classifier trained on a frozen encoder is reported (Chen and He [2020], Chen et al. [2020]). For SL, since training consists of training exactly such a classifier, an obvious metric for this lies in the validation accuracy of the model being trained. In SimSiam, however, we are not training a classifier, and we do not have an immediate relevant metric to report. Instead, a metric commonly used in SSL research as an estimate for the quality of the learned features is the accuracy of a k-nearest neighbors classifier (kNN accuracy). It is used as a proxy metric because it is relatively fast to measure and can easily be done during training as a monitor of progress. Throughout this project we will mainly use kNN accuracy to evaluate both SL and SimSiam for these reasons. For measuring kNN accuracy we use $k = 200$ and $\tau = 0.1$, as is common practice in SSL research (Chen and He [2020], Wu et al. [2018]). Perhaps somewhat surprisingly, when evaluating SimSiam we did not observe a meaningful difference in the kNN accuracy of the features produced by the encoder and those produced by the projector ($< 1\%$), and so we chose to only evaluate the kNN accuracy of the features produced by the encoder, both for simplicity and because it is the encoder features that would be used for downstream tasks such as transfer learning. We find that kNN accuracy actually very closely coincides with the validation error of the SL model, see Figure 1. On our machine it was quite costly to do kNN evaluation at every epoch, and for this reason we only evaluate every 10 epochs. This comes at a slight cost to the accuracy of our measurements, but it is a necessary measure to be able to complete our experiments in a timely fashion.

Besides kNN accuracy, in order to investigate training dynamics in SimSiam we also measure the mean feature standard deviation (std) of the l_2 -normalized outputs of the encoder computed on the unaugmented dataset with the model in evaluation mode, as well as the mean feature std of the l_2 -normalized outputs of the projector and predictor during training, i.e. on augmented training set images for that particular epoch and with model in training mode. We find that these measurements give useful insight into the training dynamics of SimSiam, and the mean feature std of the projections in particular are also used by Chen and He [2020] as a measure of collapse. These measurements, like kNN accuracy, were also computationally expensive to measure and were therefore also only measured intermittently in the same fashion.

As a quality benchmark, we evaluated the kNN accuracy of the raw pixels, and got a very surprising 40.97%, which is much higher than the random guess accuracy which would be 10%. This initially caused us to suspect that test images were leaking into the training set, but after much debugging and careful inspection of our code, we were forced to the conclusion that raw pixels are just surprisingly good on CIFAR10.

3.5 Experiment Design

We use Pytorch to implement both SL and SimSiam. All our training and experiments are performed on a single desktop PC with an AMD Ryzen 5 5600X 6-Core Processor, 32GB of RAM and an NVIDIA GeForce 3070 Ti GPU.

We first establish a baseline setup for both SL and SimSiam with respect to the data augmentation pipeline, optimizer, learning rate scheduler, training hyperparameters as well as the encoder, projector and predictor architectures. In order to observe how various training methods affect SL and SimSiam respectively, we strive to design experiments with as few differences as possible between the SL and SimSiam setups, whilst also being careful so as not to artificially hamper either of the two methods. This means ideally that SL and SimSiam share as much of their setup as possible, but if certain components of the setup either harm or make no difference to the performance of one of the methods, we either tweak or remove them entirely for simplicity. We describe the process of selecting each of the components of the baseline setup in the following subsections. Simultaneously we will motivate and summarize the final experiments that we perform and present in the Results section. See Section 3.6 for a summary of our final baseline setup.

3.5.1 Network Architecture

We use an encoder architecture similar to the ResNet (He et al. [2015]) architectures. The architecture is parametrized such that it can be scaled in width and depth, and additional features common in CNN architectures, namely batch normalization (BN) and residual connections, can be toggled on or off for experimentation purposes. The architecture is segmented into an initial layer, referred to as the stem, and four following groupings of layers, referred to as stages. Each stage consists of some number of basic blocks (see He et al. [2015]), with stages 2 and onward containing downsampling to increase the receptive field of the network. The stem is always a 3×3 convolutional layer with a stride and padding of 1, followed by BN (if enabled) and ReLU activation. The number of channels for this convolutional layer is parametrized as the `base_width`. The network architecture has an additional parameter `num_blocks` which determines the number of basic blocks contained in each stage. Each basic block contains two 3×3 convolutional

layers, each followed by BN (if enabled) and ReLU activation. For stages 2 and onward, the first convolutional layer downsamples by a factor of 2 by using a stride of 2, and the number of channels is doubled. This means that the number of channels in the final output is $\text{base_width} \cdot 2^3$. If residual connections are enabled, the output of a basic block is $x + \mathcal{F}(x)$ followed by ReLU activation, where $\mathcal{F}(x)$ denotes the output of the contents of the basic block. In the case of downsampling in the contents of the basic block, the input x is also downsampled for the residual connection using 1×1 convolutions of stride 2 to align the dimensions. If residual connections are disabled, the output of a basic block is simply $\mathcal{F}(x)$. For the final output of the encoder, an adaptive average pooling operation is applied across all channels to produce a vector of dimensionality $\text{base_width} \cdot 2^3$. Setting $\text{base_width} = 64$ and $\text{num_blocks} = 2$ with BN and residuals enabled, we get something essentially identical to the ResNet-18 architecture (pyt [2025a]) with a slightly different stem. For our baseline setup we choose to use a smaller architecture with $\text{base_width} = 32$ and $\text{num_blocks} = 1$ for faster experimentation, but we also experiment with larger values. We use the PyTorch default Kaiming uniform initialization.

For our baseline implementation of SimSiam we use 2-layer MLPs for both the projector and predictor, following the implementation of Chen and He [2020] for their experiments on the CIFAR10 dataset. Both layers in the projector have output dimensionality of $4 \cdot d$ where d denotes the dimensionality of the input (i.e. the output of the encoder). The first layer in the predictor has output dimensionality of $\frac{d}{4}$, while the second layer has output dimensionality d , where d denotes the dimensionality of the input (i.e. the output of the projector). This setup follows the recommendations of Chen and He [2020].

One of the main findings of Chen et al. [2020] is that SimCLR benefits more from wider and deeper encoder architectures than SL, i.e. the gap between SL and SimCLR closes as the encoder architecture becomes bigger. We wanted to test this finding on SimSiam, and so we perform experiments increasing the width and depth of our baseline architecture.

Residual connections are a technique for dealing with vanishing gradients, particularly in very deep network architectures. Our baseline architecture is not particularly deep, and so we omit residual connections in our baseline setup for simplicity. Since the nature of the gradient flow in SimSiam differs substantially from that in SL, we also explore how residual connections affect SL and SimSiam respectively. Specifically we hypothesize that residual connections benefit SimSiam particularly because the stop-gradient kills off half of the gradient signal during training, and so SimSiam suffers from weaker gradients compared to SL.

Inspecting Table I(d) We see that increasing the width and depth of the encoder increases the kNN accuracy of both SL and SimSiam by a similar amount, and does not seem to close the kNN accuracy gap between the two methods. We find that residual connections generally offer a small improvement to both approaches, SimSiam slightly more so than SL, especially on the larger encoder where SL actually suffers from residual connections, which we find very odd and can not offer any good explanation for.

3.5.2 Data Augmentation

For training SimSiam and later experiments with SL we implement an online data augmentation pipeline, mostly following the implementation of Chen and He [2020] in their experiments using CIFAR10. When an image is selected from this augmenting training set dataloader, a series of random data augmentations are applied with some application probability. We describe the augmentations by referring to the Pytorch functions used to compute them (pyt [2025b]). First a RandomResizedCrop is always taken with scale in the range $[0.4, 1]$. We changed these values from $[0.2, 1]$ as was used in Chen and He [2020], we will justify why later. Next ColorJitter with $\text{brightness} = 0.4$, $\text{contrast} = 0.4$, $\text{saturation} = 0.4$, $\text{hue} = 0.1$ is applied with probability 0.8. Next RandomGrayscale is applied with probability 0.2, and finally RandomHorizontalFlip is applied with probability 0.5. Once an image has been augmented, it is normalized with constant per-channel mean and std values from the entire unaugmented dataset.

It is very important that data augmentations are semantics-preserving, as they are supposed to communicate to the network the invariances of the different labels. Using the settings of SimSiam, we note that the scale range for RandomResizedCrop is as low as 0.2, which, especially in CIFAR10, we suspect could destroy the semantics of an image. We confirm this by manually inspecting the augmentation pipeline: As an example an image of an airplane can be cropped to just contain a blue sky background. We tweak this random crop range to $[0.4, 1]$ and get an approximately 5% bump in kNN accuracy on our baseline setup, and so we choose to keep this change in our augmentation pipeline.

One of the findings of Chen et al. [2020] is that SimCLR benefits from stronger data augmentation than SL. We seek to test this finding against SimSiam by experimenting with a weaker version of the data augmentation pipeline. Specifically we change the scale range of RandomResizedCrop to $[0.8, 1]$, the arguments to ColorJitter to $\text{brightness} = 0.2$, $\text{contrast} = 0.2$, $\text{saturation} = 0.2$, $\text{hue} = 0.05$ and omit RandomGrayscale. We evaluate both variants of data augmentation on our baseline setup for SL and SimSiam, and find that indeed also SimSiam benefits from stronger augmentations than are beneficial to SL. See Table I(a) for the results.

3.5.3 Optimization and Learning Rate

Due to hardware and time constraints, performing extensive hyperparameter tuning was not feasible, so we instead take hyperparameter settings from the CIFAR10 experiments of [He et al. 2015] for SL, and the CIFAR10 experiments of [Chen and He 2020] for SimSiam, and try to align them without hurting the performance of either method. Firstly for simplicity we changed the learning rate scheduler for SL from step-wise decreasing to a cosine scheduler, which is a more modern choice and also aligns with the SimSiam setup. We then reduced the batch size in SimSiam from 512 to 128 and linearly scaled the base learning rate accordingly, which did not lead to a decrease in performance. This gives us our baseline training setup for SL where we train using stochastic gradient descent (SGD) with a momentum of 0.9, a cosine learning rate scheduler with a base learning rate of 0.1, weight decay of 0.0001 and a batch size of 128. For SimSiam our baseline training setup is SGD with a momentum of 0.9, a cosine learning rate scheduler with a base learning rate of 0.0075, weight decay of 0.0005 and a batch size of 128.

We notice that the main difference between the setups used in [He et al. 2015] and [Chen and He 2020] is the base learning rate, which differ by an order of magnitude. This leads us to experiment with swapping the base learning rates between SL and SimSiam. We find, somewhat surprisingly, that our baseline SL setup is not particularly affected by this quite drastic change, while our baseline SimSiam setup suffered a significant decrease in kNN accuracy. These findings, also corroborated by the setups used in the respective papers, could suggest that SimSiam generally requires lower learning rates than SL, or may be more sensitive to learning rates. See Table 1(b) for the results.

3.5.4 Batch Normalization

[Chen and He 2020] explore the effects of BN specifically in the projector and predictor, and show that BN after all layers (including the final layer) of the projector produces the best results, while BN should only follow after the first layer in the predictor, as BN after the final layer makes training unstable. We follow these recommendations for our implementation.

BN enforces per-dimension zero mean and unit variance (over the batch) for convolutional layer outputs. Its main purpose in networks using ReLU activations is to remedy vanishing gradients. By enforcing per-dimension zero mean and unit variance BN can prevent outputs from being heavily clustered, which also relates closely to collapse conditions for SimSiam: Specifically, collapse describes a situation where outputs become increasingly clustered during training, approaching a single constant output. Therefore, we want to explore the effects of BN in SimSiam, comparing with SL. We hypothesize that BN prevents initial training conditions that can lead to collapse in SimSiam. We test this hypothesis by first running the baseline setup for both SL and SimSiam with all instances of BN removed, in both the encoder, projector and predictor. Hereafter we reintroduce BN only in the projector and observe the effects in SimSiam. The results of our experiments suggest that BN, besides alleviating vanishing gradients, actually does prevent collapse in SimSiam, see Section 4.2 for results and further discussion.

3.5.5 Nonlinear and High-Dimensional Projections

[Chen et al. 2020] find that for SimCLR, a non-linear projector improves the representation quality of the encoder. [Chen and He 2020] also use a non-linear projector for SimSiam, but additionally with a higher output dimensionality than that of the encoder. [Chen et al. 2020] specifically find that only non-linearity in the projections benefit SimCLR, and that expanding the dimensionality does not help SimCLR. This leads us to investigate the impact of the projection head in SimSiam. Based on the results of our experiments with BN, we hypothesize that because the projector (with BN, importantly) is able to isotropically scatter the embeddings, SimSiam consequently enjoys a healthier training dynamic that is farther from a collapse-like state, which then in turn improves the representation quality. The intuition is that if there exists a direction of low variance, the predictor may predict a constant (a sort of cluster center) in that direction and only suffer a small loss, and so there is no opportunity for the encoder to learn with respect to that direction, since it lies downstream from the predictor in terms of gradient signal. We test our hypothesis by running SimSiam in our baseline setup with the projector removed.

Inspecting Table 1(c) we see, quite surprisingly, that adding a projector only gives marginal improvements to kNN accuracy for both SL and SimSiam at the cost of adding a significant number of parameters to the model. As an example, in our baseline SL setup adding a 1024-d 2-layer MLP projector increases the number of parameters from 1,185,066 to 2,509,610. We get similar results if we scale the encoder as in our other experiments, so we do not believe the usefulness of the projector is determined by the scale of the encoder. We suspect rather that our choice of dataset plays a role in the necessity for a projector, which motivates further future investigation.

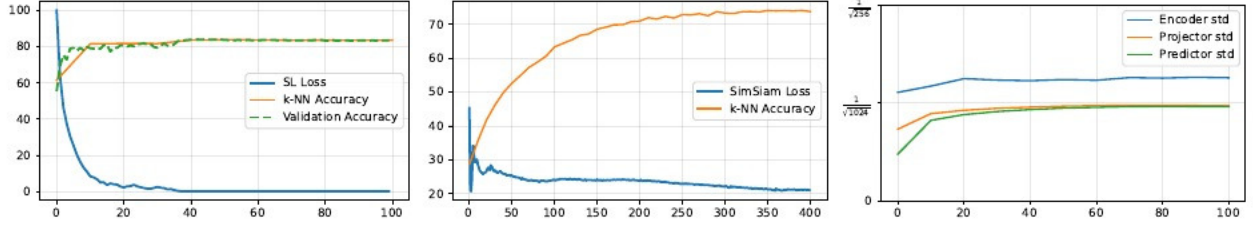


Figure 1: Baseline performance throughout training. All x-axes are number of epochs. **Left plot:** SL training loss divided by loss in first epoch and scaled by 100, validation accuracy and kNN accuracy of encoder features. **Middle plot:** SimSiam training loss offset by 1 and scaled by 100 and kNN accuracy of encoder features. **Right plot:** Mean feature std of l_2 normalized outputs of encoder in evaluation mode on unaugmented training set, and l_2 normalized outputs of projector and predictor during training.

3.6 Baseline Setup

For our baseline setup we use an encoder architecture with `base_width = 32` and `num_blocks = 1` with BN enabled and residual connections disabled. This gives us a 9-layer encoder that, when given a $32 \times 32 \times 3$ volume produces a 256 dimensional vector as the embedding. For SimSiam we additionally use as the projector a 2-layer MLP with output dimensionality of 1024 in both layers and BN in both layers. For the predictor in SimSiam we use a 2-layer MLP with output dimensionality 256 in the first layer and 1024 in the second layer, and BN only in the first layer. For both SL and SimSiam we train using stochastic gradient descent (SGD) with a batch size of 128, momentum of 0.9, and a cosine learning rate scheduler. SL uses a base learning rate of 0.1, weight decay of 0.0001. SimSiam uses a base learning rate of 0.0075, weight decay of 0.0005. SL trains for 100 epochs, SimSiam trains for 400 epochs.

4 Results

For our plots we scale and offset the training losses such that they fit on the same plot as the kNN accuracy. This preserves the overall shape of the loss curve and allows for easier comparison with the shape of the kNN accuracy curve (and saves space on the page). We normalize SL loss by its initial value (i.e. the loss in the first epoch) and scale by 100 to get values in the range $[0, 100]$. We offset SimSiam loss by 1 and scale by 100 to get values in the range $[0, 100]$.

4.1 Baseline Performance

Inspecting Figure 1 we can see what healthy training looks like for both SL and SimSiam. For SL we see that within the first 40 epochs the training loss rapidly decreases to near-zero and plateaus. For the validation and kNN accuracies, which align quite closely in value throughout training, we observe the inverse curvature, that they rapidly increase and hereafter plateau. In other words, we see that as the loss decreases, validation and kNN accuracy increase. We find it curious that in our baseline SL setup the initial validation and kNN accuracies are as high as approximately 60% in a completely untrained network. For SimSiam we can observe a somewhat different pattern: Initially in the first 20 or so epochs, the training loss oscillates and then stabilizes around some value, whereafter it very slowly decreases throughout training. Despite the loss remaining relatively stationary, the kNN accuracy nonetheless significantly increases throughout training. This is an example of the training loss in SSL not being directly correlated to the performance on a downstream task, in this case kNN classification. It is also apparent that SimSiam needs many more epochs to finish training than SL, here around an order of magnitude, as SL plateaus after approximately 40 epochs and SimSiam after approximately 400 epochs. In the Right plot we see that the mean std of the 1024-d l_2 normalized outputs of the projector and predictor quickly approaches $1/\sqrt{1024}$ as training progresses, which indicates that they are isotropically scattered on the unit-hypersphere (Chen and He (2020)). This discourages collapse and is at least in part due to BN, as our other experiments indicate. We also see that the mean std of the 256-d l_2 normalized encoder features increases slightly as training progresses, but does not come nearly as close to its respective upper bound of $1/\sqrt{256}$ as the projector and predictor outputs.

4.2 Effects of Batch Normalization

Inspecting Figure 2 we can see how removing BN affects both SL and SimSiam. In the case of SL, we see that the training loss is completely flatlined and validation accuracy is similarly stationary at an accuracy of approximately 10%, which corresponds to random guessing in CIFAR10. This heavily indicates that SL is affected by vanishing gradients

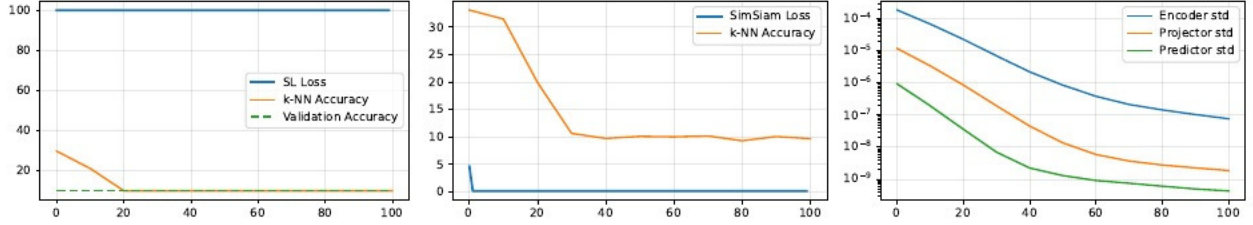


Figure 2: Performance of SL and SimSiam without BN, same measurements as in Figure 1. Right plot has log-scale y-axis for visual clarity.

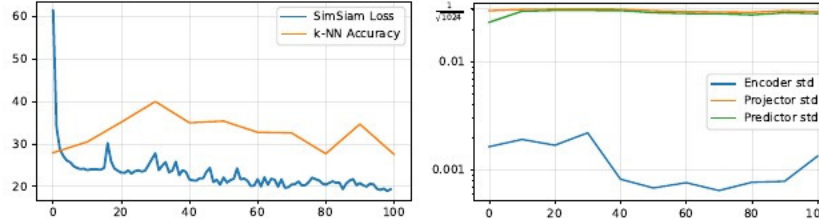


Figure 3: Performance of SimSiam with BN only in the projector, same measurements as in Figure 1. Right plot has log-scale y-axis for visual clarity.

and is consequently unable to learn during training. It is curious that despite the model evidently undergoing minimal change, the initial kNN accuracy of approximately 30% still manages to degrade to around 10% as well. In the case of SimSiam we see that the training loss starts out at a very low initial value and immediately becomes extremely close to the minimum possible value and remains so throughout training. Simultaneously we see kNN accuracy rapidly declining from its initial value to around 10%. We see heavy indications of collapse when inspecting the mean std values, all starting out at values several orders of magnitude lower than baseline and rapidly further decreasing several orders of magnitude as training progresses. We believe the following training dynamic is taking place: Because of a complete lack of BN, the initial outputs of the projector are heavily clustered around some constant point, and the predictor outputs even more so, having gone through even more layers of the network. This causes the very low initial training loss and essentially initializes the network in an already collapsed state, trapping the network in a part of the loss landscape where the only way to improve is to further cluster the outputs, which only worsens the collapse, hence why we see the further decrease of the mean std values.

This interpretation is supported by the results in Figure 3 where we reintroduce BN only in the projector. We see that the mean std values of the projector and predictor indicate, as in our baseline, that outputs are isotropically scattered, which in turn stabilizes the mean std of the encoder features. We also see that the initial loss is much higher and the loss curves resembles our baseline. Despite this we can also see that the kNN accuracy of the features are somewhat unstable and do not generally improve as training progresses, so we may conclude that BN in the projector, while preventing collapse, is not enough to facilitate useful learning. This could possibly be explained by the encoder itself still suffering from vanishing gradients.

4.3 Additional notes

As a final experiment and an attempt to see the kNN accuracy gap between SL and SimSiam in their respective "best" configurations, we use an encoder with `base_width = 64` `num_blocks = 4` and residual connections enabled, train SL with our weaker variant of data augmentation for 200 epochs and train SimSiam with no projector for 800 epochs. We measure a maximum kNN accuracy of 91.83 % and 83.03 % for SL and SimSiam respectively, which we interpret as suggesting diminishing returns on scaling the encoder architecture for both methods as a means of increasing kNN accuracy. However, it is possibly also in part due to a suboptimal setting of hyperparameters that we do not see a higher increase.

Our SL result of 91.83 %, here with a 33-layer encoder, are very comparable with the 92.49 % that He et al. [2015] achieve with a 32-layer ResNet architecture on CIFAR10. Despite essentially copying the setup of Chen and He [2020] for some of our experiments, we were unable to match their result of 91.8 % kNN accuracy for SimSiam on CIFAR10. This could be due to omitted implementation details that we unknowingly deviate from, or of course faults in our implementation that we are unaware of. If we should further pursue the goal of closing the gap between SL and SimSiam on CIFAR10 specifically, we would turn to tuning the training hyperparameters as well as the data augmentation.

(a) Effects of data augmentation strength				(b) Effects of base learning rate			
Method	Epochs	Augment.	kNN (%)	Method	Epochs	Base LR	kNN (%)
SL	100	None	83.83	SL	100	0.1	83.83
SL	100	Weak	86.20	SL	100	0.0075	83.41
SL	100	Strong	83.28	SimSiam	100	0.1	56.37
SimSiam	100	Weak	47.42	SimSiam	100	0.0075	63.32
SimSiam	100	Strong	63.32				

(c) Effects of projector						(d) Effects of encoder architecture					
Method	Epochs	Projector	kNN (%)	Method	Epochs	Base W.	Blocks	Res.	kNN (%)	Method	Epochs
SL	100	None	83.83	SL	100	32	1	No	83.83	SL	100
SL	100	2-layer MLP w/o BN	84.60	SL	100	32	1	Yes	83.96	SL	100
SimSiam	400	None	74.08	SL	100	64	2	No	88.54	SL	100
SimSiam	400	2-layer MLP w/ BN	74.52	SL	100	64	2	Yes	85.11	SimSiam	400
				SimSiam	400	32	1	No	74.08	SimSiam	400
				SimSiam	400	32	1	Yes	74.90	SimSiam	400
				SimSiam	400	64	2	No	78.30	SimSiam	400
				SimSiam	400	64	2	Yes	80.30		

Table 1: Summary of kNN accuracies (highest measured during training) for different experiments. Columns describe the method used, number of epochs trained, deviations in setup from baseline and resulting kNN accuracy. In the instances where SimSiam was training for fewer epochs than baseline i.e. 400 epochs, the learning rate scheduler was kept the same, i.e. with $\tau_{\max} = 400$ to enable for comparison with baseline kNN accuracy after 100 epochs.

5 Discussion

As a way of exploring the differences between supervised and self-supervised representation learning, We implement standard cross-entropy SL and SimSiam, and perform various experiments to see how they differ in practice and each are affected by various methods for training CNNs. We also investigate related findings of [Chen et al. \[2020\]](#) for SimCLR, another Siamese SSL method bearing resemblances to SimSiam, and test if they apply to SimSiam. Generally we observe for our implementation that on the same underlying encoder architecture, SimSiam is able to achieve kNN accuracies approximately 8-9% lower than SL, when given a suitable setting of training hyperparameters, sufficiently strong data augmentation and significantly more (approximately an order of magnitude more) epochs to train. We find these observations to hold across multiple scalings of our encoder architecture, both width- and depth-wise, and so our findings for SimSiam do not align with those of [Chen et al. \[2020\]](#) for SimCLR, that wider and deeper encoders close the performance gap. Our findings do align with [Chen et al. \[2020\]](#) with regards to data augmentations and training length: SimSiam, like SimCLR, benefits greatly from data augmentations that are stronger than those beneficial to SL, and requires significantly longer training. We investigate the effects of BN in SimSiam and find that it is an essential component to avoid collapse and facilitate productive training, and we propose a potential explanation for why that is the case, taking into account the unique training dynamics specific to SimSiam. We also discover that the projector in SimSiam is not always necessary to learn useful features, and it can be a significant save in model size to omit it. It is possible that our choice of dataset in its simplicity is simply not conducive to illustrating the utility of a high-dimensional non-linear projector, or the full potential effects of scaling the encoder architecture.

A limitation of our project is the training times on our available hardware (approximately 2 hours for baseline SL and 8 hours for baseline SimSiam), which under the time constraints of this project prevents us in multiple ways from strengthening our results: Firstly, it simply hinders us in the number of experiments we can perform, and thus the degree to which we can explore different possible configurations of our implementation. It also prevents us from performing repeated trials of our experiments, which may also affect our results. [Chen and He \[2020\]](#) generally found well-parametrized SimSiam training to be quite stable (linear evaluation std less than 0.1 across 5 trials), but if any of our experimental setups cause training to be unstable, we are not able to detect this because we only do a single trial. This effect may very well explain some of our more confusing results. Another consequence is that we are unable to perform extensive training-hyperparameter tuning, which may obscure our results e.g. across different encoder architectures, due to our fixed choice of base learning rate and weight decay possibly favoring certain setups. It also further limits us in the extent to which we are able to scale the encoder architectures, restricting us to relatively small architectures. Our choice of the relatively simple CIFAR10 as our dataset could also mean that our findings will not generalize to more complex datasets.

References

- Xinlei Chen and Kaiming He. Exploring simple siamese representation learning, 2020. URL <https://arxiv.org/abs/2011.10566>.
- Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations, 2020. URL <https://arxiv.org/abs/2002.05709>.
- Mathilde Caron, Ishan Misra, Julien Mairal, Priya Goyal, Piotr Bojanowski, and Armand Joulin. Unsupervised learning of visual features by contrasting cluster assignments, 2021. URL <https://arxiv.org/abs/2006.09882>.
- Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning, 2020. URL <https://arxiv.org/abs/2006.07733>.
- Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. Technical report, University of Toronto, Toronto, Ontario, 2009. URL <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015. URL <http://arxiv.org/abs/1512.03385>.
- Zhirong Wu, Yuanjun Xiong, Stella Yu, and Dahua Lin. Unsupervised feature learning via non-parametric instance-level discrimination, 2018. URL <https://arxiv.org/abs/1805.01978>.
- Pytorch documentation: Resnet18. <https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html> 2025a. Accessed: 2025-12-04.
- Pytorch documentation: Transforms. <https://docs.pytorch.org/vision/main/transforms.html> 2025b. Accessed: 2025-12-04.